

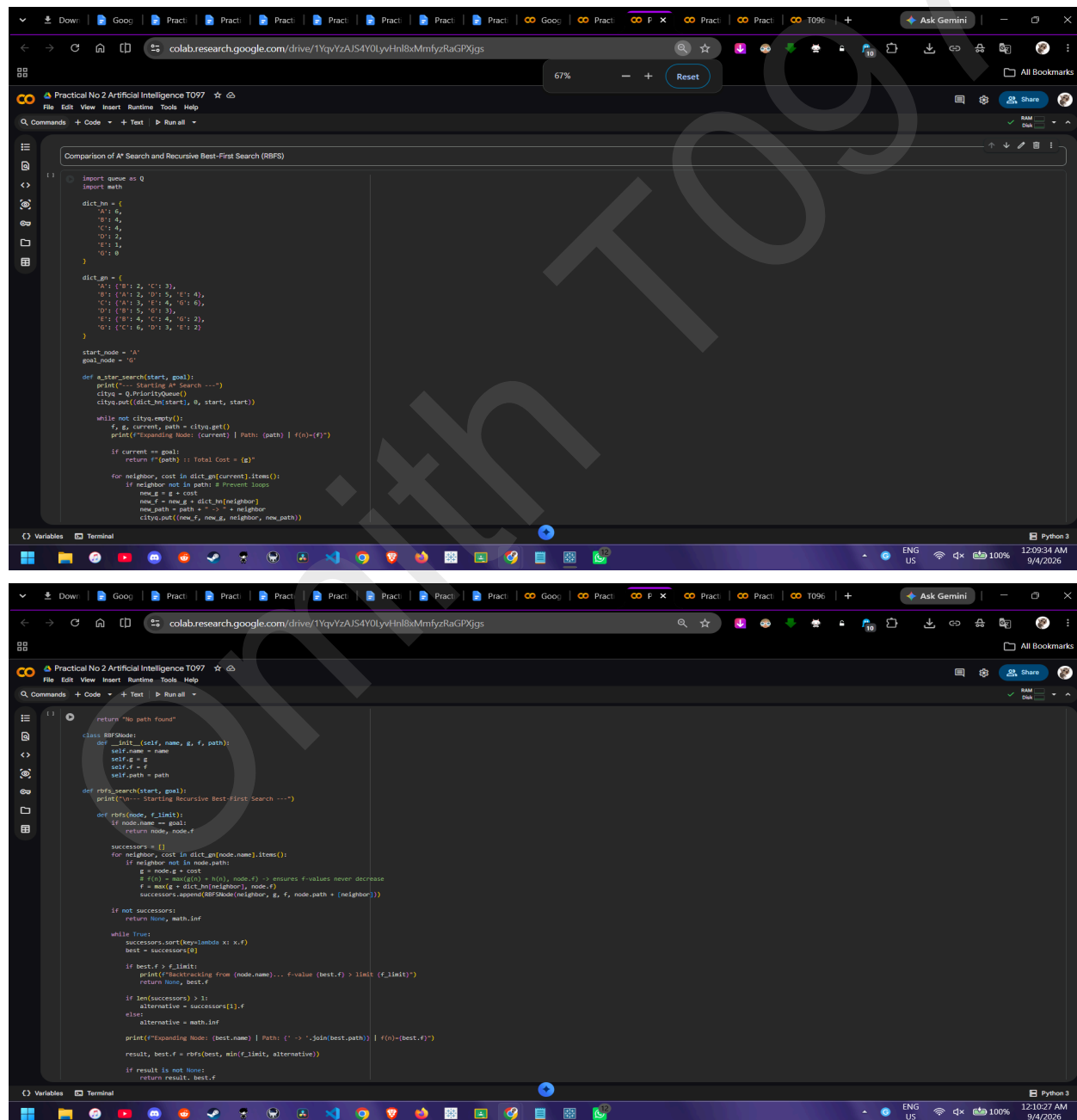
Practical No 2

Aim: A* Search and Recursive Best-First Search

- Implement the A* Search algorithm for solving a pathfinding problem.
- Implement the Recursive Best-First Search algorithm for the same problem.
- Compare the performance and effectiveness of both algorithms.

Comparison of A* Search and Recursive Best-First Search (RBFS)

Code:



The image displays two screenshots of a Google Colab notebook titled "Practical No 2 Artificial Intelligence T097". The first screenshot shows the A* Search algorithm implementation. It includes a graph definition with nodes A through G and their neighbors, a priority queue, and a search function that expands nodes based on their total cost (g + h). The second screenshot shows the Recursive Best-First Search (RBFS) algorithm implementation. It includes a class for RBFSNode, a search function that recursively explores nodes, and a function to return the best path found. Both screenshots show the code in a dark-themed editor with a terminal at the bottom.

```
1.1 import queue as Q
import math

dict_hn = {
    'A': 6,
    'B': 4,
    'C': 4,
    'D': 2,
    'E': 3,
    'G': 0
}

dict_gn = {
    'A': ('B': 2, 'C': 3),
    'B': ('A': 2, 'D': 3, 'E': 4),
    'C': ('A': 3, 'E': 4, 'G': 0),
    'D': ('B': 3, 'G': 3),
    'E': ('B': 4, 'C': 4, 'G': 2),
    'G': ('C': 4, 'D': 3, 'E': 2)
}

start_node = 'A'
goal_node = 'G'

def a_star_search(start, goal):
    print("--- Starting A* Search ---")
    cityq = Q.PriorityQueue()
    cityq.put((dict_hn[start], 0, start, start))

    while not cityq.empty():
        f, g, current, path = cityq.get()
        print(f"Expanding Node: {current} | Path: {path} | f(x)=f(g)")

        if current == goal:
            return f(path)  # Total Cost = g!

        for neighbor, cost in dict_gn[current].items():
            if neighbor not in path: # prevent loop
                new_g = g + cost
                new_f = new_g + dict_hn[neighbor]
                new_path = path + ' -> ' + neighbor
                cityq.put((new_f, new_g, neighbor, new_path))

return "No path found"

class RBFSNode:
    def __init__(self, name, g, f, path):
        self.name = name
        self.g = g
        self.f = f
        self.path = path

def rbfs_search(start, goal):
    print("--- Starting Recursive Best-First Search ---")
    def rbfs(node, f_limit):
        if node.name == goal:
            return node.name, node.f

        successors = []
        for neighbor, cost in dict_gn[node.name].items():
            if neighbor not in node.path:
                g = node.g + cost
                f = max(g + dict_hn[neighbor], node.f) # ensures f-values never decrease
                successors.append(RBFSNode(neighbor, g, f, node.path + (neighbor)))

        if not successors:
            return None, math.inf

        while True:
            successors.sort(key=lambda x: x.f)
            best = successors[0]

            if best.f > f_limit:
                print(f"Backtracking from {node.name}... f-value {best.f} > limit {f_limit}")
                return None, best.f

            if len(successors) > 1:
                alternative = successors[1].f
            else:
                alternative = math.inf

            print(f"Expanding Node: {best.name} | Path: '{' -> '.join(best.path)}' | f(x)=(best.f)")
            result, best.f = rbfs(best, min(f_limit, alternative))

            if result is not None:
                return result, best.f

    return rbfs(RBFSNode(start, dict_hn[start], 0, start), dict_hn[start])
```

SHETH LUJ AND SIR MV COLLEGE

Artificial Intelligence

```
start_node_obj = RBFSNode(start, 0, dict_hn(start), [start])
result_node, _ = rbfh(start_node_obj, math.inf)

if result_node:
    return f'({start} -> {result_node.path}) :: Total Cost = {result_node.g}'
return 'No path found'

if __name__ == '__main__':
    a_star_result = a_star_search(start_node, goal_node)
    print(f'([FINAL A* RESULT] Optimal Path: {a_star_result})')

    rbfh_result = rbfh_search(start_node, goal_node)
    print(f'([FINAL RBFS RESULT] Optimal Path: {rbfh_result})')
```

Recursive Best-First Search (RBFS) for Map Navigation

```
import matplotlib.pyplot as plt

graph = {
    'MWLU': {'HIDC': 12, 'Tolani': 14},
    'HIDC': {'SEEPZ': 6, 'Marol': 10},
    'Tolani': {'Marol': 13},
    'SEEPZ': {'Chandivall': 9},
    'Marol': {'Chandivall': 6},
    'Chandivall': {'Powai_Lake': 3},
    'Powai_Lake': {}
}

heuristics = {
    'MWLU': 22, 'HIDC': 14, 'Tolani': 13,
    'SEEPZ': 9, 'Marol': 7, 'Chandivall': 2, 'Powai_Lake': 0
}

coords = {
    'MWLU': (2, 0), 'HIDC': (1, 4.5), 'Tolani': (3, 4.5),
    'SEEPZ': (1, 3), 'Marol': (3, 3), 'Chandivall': (2, 1.5), 'Powai_Lake': (2, 0)
}

def rbfh_search(start, goal):
    success, path, cost, _ = rbfh(start, goal, g=0, f_limit=float('inf'), path=[start])
    return path, cost
```

Output:

```
rbfh_result = rbfh_search(start_node, goal_node)
print(f'([FINAL RBFS RESULT] Optimal Path: {rbfh_result})')
```

--- Starting A* Search ---
Expanding Node: A | Path: A -> B | f(n)=6
Expanding Node: B | Path: A -> C | f(n)=6
Expanding Node: C | Path: A -> B -> E | f(n)=7
Expanding Node: E | Path: A -> C -> E | f(n)=8
Expanding Node: G | Path: A -> B -> E -> G | f(n)=8
[FINAL A* RESULT] Optimal Path: A -> B -> E -> G :: Total Cost = 8

--- Starting Recursive Best-First Search ---
Expanding Node: B | Path: A -> B | f(n)=6
Expanding Node: E | Path: A -> B -> E | f(n)=7
Backtracking from E... f-value 8 > limit 7
Expanding Node: C | Path: A -> C | f(n)=7
Expanding Node: E | Path: A -> C -> E | f(n)=8
Backtracking from E... f-value 9 > limit 8
Backtracking from C... f-value 9 > limit 8
Expanding Node: B | Path: A -> B | f(n)=8
Expanding Node: E | Path: A -> B -> E | f(n)=8
Expanding Node: G | Path: A -> B -> E -> G | f(n)=8
[FINAL RBFS RESULT] Optimal Path: A -> B -> E -> G :: Total Cost = 8

Recursive Best-First Search (RBFS) for Map Navigation

```
import matplotlib.pyplot as plt

graph = {
    'MWLU': {'HIDC': 12, 'Tolani': 14},
    'HIDC': {'SEEPZ': 6, 'Marol': 10},
    'Tolani': {'Marol': 13},
    'SEEPZ': {'Chandivall': 9},
    'Marol': {'Chandivall': 6},
    'Chandivall': {'Powai_Lake': 3},
    'Powai_Lake': {}
}

heuristics = {
```

SHETH LUJ AND SIR MV COLLEGE
Artificial Intelligence

Recursive Best-First Search (RBFS) for Map Navigation
Code:

```
Recursive Best-First Search (RBFS) for Map Navigation

import matplotlib.pyplot as plt

graph = {
    'NWU': {'HDC': 12, 'Tolani': 16},
    'HDC': {'SEEPZ': 6, 'Marol': 10},
    'Tolani': {'Marol': 13},
    'SEEPZ': {'Chandivall': 9},
    'Marol': {'Chandivall': 6},
    'Chandivall': {'Powai_Lake': 3},
    'Powai_Lake': {}
}

heuristics = {
    'NWU': 22, 'HDC': 14, 'Tolani': 13,
    'SEEPZ': 9, 'Marol': 7, 'Chandivall': 2, 'Powai_Lake': 0
}

coords = {
    'NWU': (2, 0), 'HDC': (1, 4.5), 'Tolani': (3, 4.5),
    'SEEPZ': (1, 3), 'Marol': (3, 3), 'Chandivall': (2, 1.5), 'Powai_Lake': (2, 0)
}

def rbfs_search(start, goal):
    success, path, cost, _ = rbfs(start, goal, g=0, f_limit=float('inf'), path=[start])
    return path, cost

def rbfs(node, goal, g, f_limit, path):
    if node == goal:
        return True, path, g, 0
    neighbors = graph.get(node, {})
    if not neighbors:
        return False, [], 0, float('inf')
    successors = []
    for neighbor, distance in neighbors.items():
        if neighbor not in path:
            next_g = g + distance
            next_f = max(next_g, heuristics[neighbor], g + heuristics[node])
            successors.append((next_f, neighbor, next_g))
    if not successors:
        return False, [], 0, float('inf')
```

```
while True:
    successors.sort(key=lambda x: x[0])
    best = successors[0]
    if best[0] > f_limit:
        return False, [], 0, best[0]
    alternative_f = successors[1][0] if len(successors) > 1 else float('inf')
    success, result_path, total_g, returned_f = rbfs(
        best[1], goal, best[2], min(f_limit, alternative_f), path + [best[1]]
    )
    best[0] = returned_f
    if success:
        return True, result_path, total_g, returned_f

path, total_dist = rbfs_search('NWU', 'Powai_Lake')
print("Optimal RBFS Path: " + ' -> '.join(path) + " (Total Dist: %s)" % total_dist)

plt.figure(figsize=(8, 7))
for node, neighbors in graph.items():
    x1, y1 = coords[node]
    for neighbor, dist in neighbors.items():
        x2, y2 = coords[neighbor]
        is_path = node in path and neighbor in path and path.index(neighbor) == path.index(node) + 1
        color, width = ('blue', 7) if is_path else ('red', 1.5)
        plt.plot([x1, x2], [y1, y2], color=color, linewidth=width, zorder=1)
        plt.text((x1+x2)/2, (y1+y2)/2, f"(distance: {dist})", color='red', fontsize=9, ha='center')

for node, (x, y) in coords.items():
    color = 'white' if node in path else 'black'
    plt.scatter(x, y, color=color, s=700, zorder=2)
    plt.text(x, y, f"{node}({heuristics[node]})", ha='center', va='center', color='white', fontsize=8, fontweight='bold')

plt.title("RBFS Shortest Route Map (Total: %s)" % total_dist)
plt.axis('off')
plt.show()
```

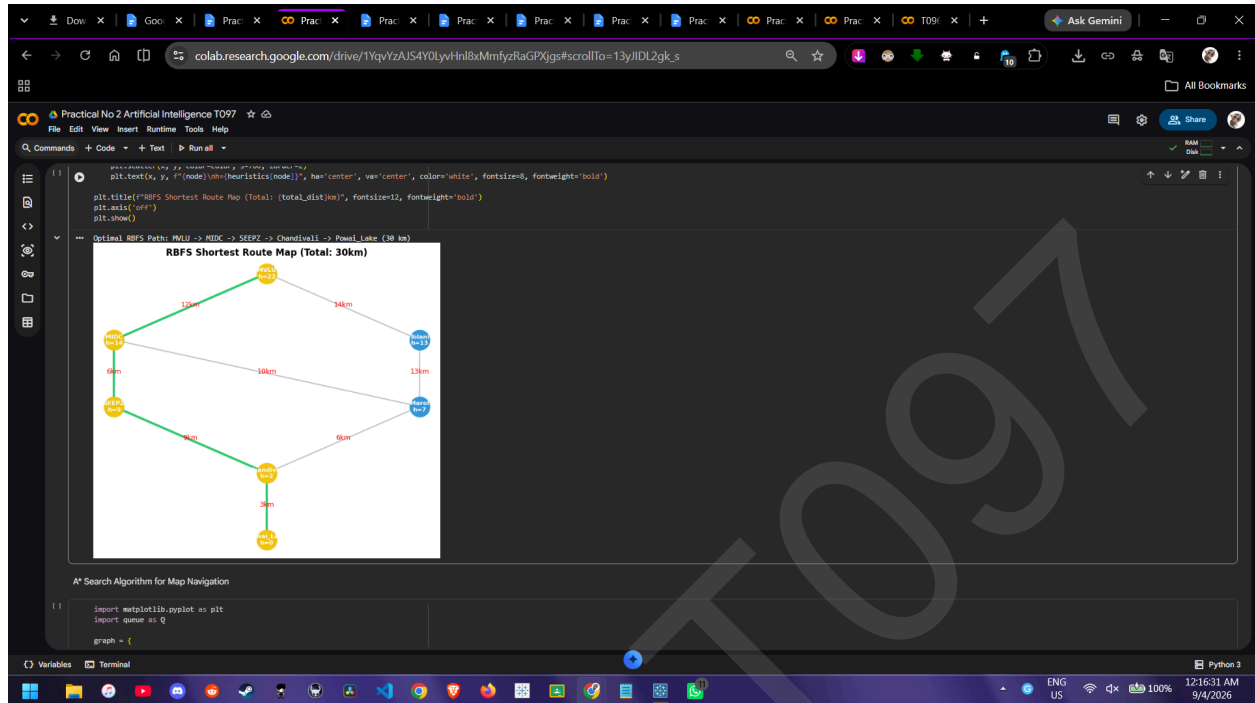
A* Search Algorithm for Map Navigation

```
import matplotlib.pyplot as plt
import sys as s
```

SHETH LUJ AND SIR MV COLLEGE

Artificial Intelligence

Output:



A* Search Algorithm for Map Navigation

Code:

```
A* Search Algorithm for Map Navigation

import matplotlib.pyplot as plt
import queue as Q

graph = {
    'MLU': ('MIDC': 12, 'Tolani': 14),
    'MIDC': ('SEEP2': 6, 'Harol': 18),
    'Tolani': ('Harol': 13),
    'SEEP2': ('Chandivall': 9),
    'Harol': ('Chandivall': 8),
    'Chandivall': ('Powal_Lake': 3),
    'Powal_Lake': {}
}

heuristics = {
    'MLU': 22, 'MIDC': 14, 'Tolani': 13,
    'SEEP2': 9, 'Harol': 7, 'Chandivall': 2, 'Powal_Lake': 0
}

coords = {
    'MLU': (2, 6), 'MIDC': (1, 4.5), 'Tolani': (3, 4.5),
    'SEEP2': (1, 3), 'Harol': (3, 3), 'Chandivall': (2, 1.5), 'Powal_Lake': (2, 0)
}

def a_star_search(start, goal):
    open_list = Q.PriorityQueue()
    open_list.put((heuristics[start], 0, start, [start]))

    while not open_list.empty():
        f_score, g_score, current_node, path = open_list.get()

        if current_node == goal:
            return path, g_score

        neighbors = graph.get(current_node, [])
        for neighbor, distance in neighbors.items():
            if neighbor not in path:
                new_g = g_score + distance
                new_f = new_g + heuristics[neighbor]

                new_path = list(path)
                new_path.append(neighbor)
                open_list.put((new_f, new_g, neighbor, new_path))
```

SHETH LUJ AND SIR MV COLLEGE

Artificial Intelligence

```
return [], 0

path, total_dist = a_star_search('MULU', 'Pomal Lake')
print("Optimal A* Path: {} -> {}".format(path, (total_dist) km))

plt.figure(figsize=(8, 7))
for node, neighbors in graph.items():
    x1, y1 = coords(node)
    for neighbor, dist in neighbors.items():
        x2, y2 = coords(neighbor)
        is_path = node in path and neighbor in path and path.index(neighbor) == path.index(node) + 1
        color, width = ('#d62728', 3) if is_path else ('black', 1.5)
        plt.plot(x1, x2, [y1, y2], color=color, linewidth=width, border=1)
        plt.text((x1+x2)/2, (y1+y2)/2, f"({dist})km", color='red', fontsize=8, ha='center')

for node, (x, y) in coords.items():
    color = '#f1c40f' if node in path else '#4682b4'
    plt.scatter(x, y, color=color, s=700, zorder=2)
    plt.text(x, y, f"({node})km", ha='center', va='center', color='white', fontsize=8, fontweight='bold')

plt.title(f"A* Shortest Route Map (Total: {total_dist}km)", fontsize=12, fontweight='bold')
plt.axis('off')
plt.show()
```

Output:

